

```
from google.colab import files
uploaded_files=files.upload()
```

No file chosen Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.
Saving 11) Powerplant dataset.csv to 11) Powerplant dataset.csv

```
import pandas as pd
import numpy as np
```

```
df=pd.read_csv("11) Powerplant dataset.csv")
```

```
df.head()
```

	AT	V	AP	RH	PE
0	8.34	40.77	1010.84	90.01	480.48
1	23.64	58.49	1011.40	74.20	445.75
2	29.74	56.90	1007.15	41.91	438.76
3	19.07	49.69	1007.22	76.79	453.09
4	11.80	40.66	1017.13	97.20	464.43

```
df.isnull().sum()
```

	0
AT	0
V	0
AP	0
RH	0
PE	0

dtype: int64

```
x=df.drop("PE",axis=1)
y=df["PE"]
```

```
from sklearn.model_selection import train_test_split
x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=0.2,random_state=42)
```

y_test

	PE
2513	433.27
9411	438.16
8745	458.42
9085	480.82
4950	441.41
...	...
7204	456.70
1599	438.04
5697	467.80
350	437.14
6210	456.78

1914 rows × 1 columns

dtype: float64

```
from sklearn.preprocessing import StandardScaler
sc=StandardScaler()
x_train_scaled=sc.fit_transform(x_train)
x_test_scaled=sc.transform(x_test)
```

x_test_scaled

```
array([[ 1.34499288,  0.23869298, -1.28658067, -1.10532538],
       [ 0.81095912,  1.36269098, -0.74140656,  0.26485915],
       [-0.2437241 , -0.73900436,  1.99970178, -0.19713193],
       ...,
       [-0.67068342, -1.15902881, -0.29951077, -0.10651852],
       [ 1.31420898,  1.33752097, -0.87346737, -0.44288647],
       [-0.2611237 , -0.27021304,  0.37433797,  1.10646548]])
```

```
import torch
import torch.nn as nn
x_train_tensor=torch.tensor(x_train_scaled, dtype=torch.float32)
```

```
y_train_tensor=torch.tensor(y_train.values, dtype=torch.float32).view(-1,1)
x_test_tensor=torch.tensor(x_test_scaled, dtype=torch.float32)
y_test_tensor=torch.tensor(y_test.values, dtype=torch.float32).view(-1,1)
```

```
from torch.utils.data import TensorDataset, DataLoader
train_dataset=TensorDataset(x_train_tensor, y_train_tensor)
test_dataset=TensorDataset(x_test_tensor, y_test_tensor)
train_loader=DataLoader(train_dataset, batch_size=32, shuffle=True)
test_loader=DataLoader(test_dataset, batch_size=32, shuffle=False)
```

```
class ANN(nn.Module):
    def __init__(self):
        super(ANN, self).__init__()
        self.model=nn.Sequential(
            nn.Linear(x_train.shape[1],6),
            nn.ReLU(),
            nn.Linear(6,6),
            nn.ReLU(),
            nn.Linear(6,1)
        )
    def forward(self,x):
        return self.model(x)
```

```
import torch.optim as optim
model=ANN()
loss_fn=nn.MSELoss()
optimizer=torch.optim.Adam(model.parameters(), lr=0.001)
```

```
epochs=100
train_loss = []
val_loss = []

for epoch in range(epochs):
    model.train()
    running_loss=0.0
    for xb ,yb in train_loader:
        output=model(xb)
        loss=loss_fn(output,yb)
        loss.backward()
        optimizer.step()
        optimizer.zero_grad()
        running_loss += loss.item()
    epoch_train_loss=running_loss/len(train_loader)
    train_loss.append(epoch_train_loss)

    running_val_loss=0.0
```

```

model.eval()
with torch.no_grad():
    for xb,yb in test_loader:
        output=model(xb)
        loss=loss_fn(output,yb)
        running_val_loss+=loss.item()
epoch_val_loss=running_val_loss/len(test_loader)
val_loss.append(epoch_val_loss)
print(f"epoch = {epoch+1}/{epochs} ==> train_loss={epoch_train_loss} & val_loss={epoch_val_loss}")

```

```

epoch = 1/100 ==> train_loss=20.32154163916906 & val_loss=20.35137769381205
epoch = 2/100 ==> train_loss=20.253879298766453 & val_loss=18.2729154586792
epoch = 3/100 ==> train_loss=20.320416710774104 & val_loss=18.697818692525228
epoch = 4/100 ==> train_loss=20.188088264067968 & val_loss=18.449610694249472
epoch = 5/100 ==> train_loss=20.04450253844261 & val_loss=18.354059521357218
epoch = 6/100 ==> train_loss=20.338211965560912 & val_loss=18.610513456662495
epoch = 7/100 ==> train_loss=20.305546971162162 & val_loss=18.89880755742391
epoch = 8/100 ==> train_loss=20.215259079138438 & val_loss=18.314164384206137
epoch = 9/100 ==> train_loss=20.33372118870417 & val_loss=18.391645375887553
epoch = 10/100 ==> train_loss=20.224528274933498 & val_loss=18.460481468836466
epoch = 11/100 ==> train_loss=20.20178322593371 & val_loss=19.612822167078654
epoch = 12/100 ==> train_loss=20.343071154753368 & val_loss=18.615400552749634
epoch = 13/100 ==> train_loss=20.209175976117454 & val_loss=18.563512142499288
epoch = 14/100 ==> train_loss=20.20010462999344 & val_loss=19.81494336128235
epoch = 15/100 ==> train_loss=20.213571474949518 & val_loss=18.84160935084025
epoch = 16/100 ==> train_loss=20.2782918771108 & val_loss=18.4895675500234
epoch = 17/100 ==> train_loss=20.22131499449412 & val_loss=18.41648384730021
epoch = 18/100 ==> train_loss=20.109883133570353 & val_loss=18.518868239720664
epoch = 19/100 ==> train_loss=20.1509548664093 & val_loss=18.99768354098002
epoch = 20/100 ==> train_loss=20.262629425525667 & val_loss=18.91492379506429
epoch = 21/100 ==> train_loss=20.250221866369248 & val_loss=19.188387870788574
epoch = 22/100 ==> train_loss=20.18092025121053 & val_loss=18.324144506454466
epoch = 23/100 ==> train_loss=20.243763967355093 & val_loss=18.218124794960023
epoch = 24/100 ==> train_loss=20.180549544095992 & val_loss=18.40488924185435
epoch = 25/100 ==> train_loss=20.20874285697937 & val_loss=18.36771281560262
epoch = 26/100 ==> train_loss=20.272541761398315 & val_loss=20.188463735580445
epoch = 27/100 ==> train_loss=20.199559279282887 & val_loss=18.435809993743895
epoch = 28/100 ==> train_loss=20.25925352970759 & val_loss=18.72160946528117
epoch = 29/100 ==> train_loss=20.114684571822483 & val_loss=19.923671102523805
epoch = 30/100 ==> train_loss=20.21393023530642 & val_loss=18.43437655766805
epoch = 31/100 ==> train_loss=20.144987815618514 & val_loss=18.43111923535665
epoch = 32/100 ==> train_loss=20.396105162302653 & val_loss=19.392127513885498
epoch = 33/100 ==> train_loss=20.097123537460963 & val_loss=19.36724654833476
epoch = 34/100 ==> train_loss=20.214361063639323 & val_loss=18.33411439259847
epoch = 35/100 ==> train_loss=20.310120062033334 & val_loss=18.69801286061605
epoch = 36/100 ==> train_loss=20.310270929336546 & val_loss=18.21835862795512
epoch = 37/100 ==> train_loss=20.280018230279286 & val_loss=21.508640098571778
epoch = 38/100 ==> train_loss=20.3499333580335 & val_loss=18.71581506729126
epoch = 39/100 ==> train_loss=20.24598395427068 & val_loss=19.372609146436055
epoch = 40/100 ==> train_loss=20.262531328201295 & val_loss=18.341032568613688
epoch = 41/100 ==> train_loss=20.14157089392344 & val_loss=18.27695693174998
epoch = 42/100 ==> train_loss=20.289097481966017 & val_loss=18.909029372533162
epoch = 43/100 ==> train_loss=20.184208365281425 & val_loss=18.61719455718994

```

```

epoch = 44/100 ==> train_loss=20.258106064796447 & val_loss=18.332477156321207
epoch = 45/100 ==> train_loss=20.545294052362443 & val_loss=19.127161852518718
epoch = 46/100 ==> train_loss=20.179528148969016 & val_loss=19.200066129366558
epoch = 47/100 ==> train_loss=20.21192639072736 & val_loss=18.57515269915263
epoch = 48/100 ==> train_loss=20.165893685817718 & val_loss=18.204120723406472
epoch = 49/100 ==> train_loss=20.19328301946322 & val_loss=18.684415435791017
epoch = 50/100 ==> train_loss=20.321081791321436 & val_loss=18.41998119354248
epoch = 51/100 ==> train_loss=20.15610217253367 & val_loss=18.475355593363442
epoch = 52/100 ==> train_loss=20.169798572858173 & val_loss=18.47406636873881
epoch = 53/100 ==> train_loss=20.033458117643992 & val_loss=18.474913342793783
epoch = 54/100 ==> train_loss=20.302886712551118 & val_loss=18.813852326075235
epoch = 55/100 ==> train_loss=20.346318598588308 & val_loss=18.854311323165895
epoch = 56/100 ==> train_loss=20.24673916896184 & val_loss=18.67265577316284
epoch = 57/100 ==> train_loss=20.054988531271615 & val_loss=18.244885428746542
epoch = 58/100 ==> train_loss=20.026010892155822 & val_loss=18.040808214476014

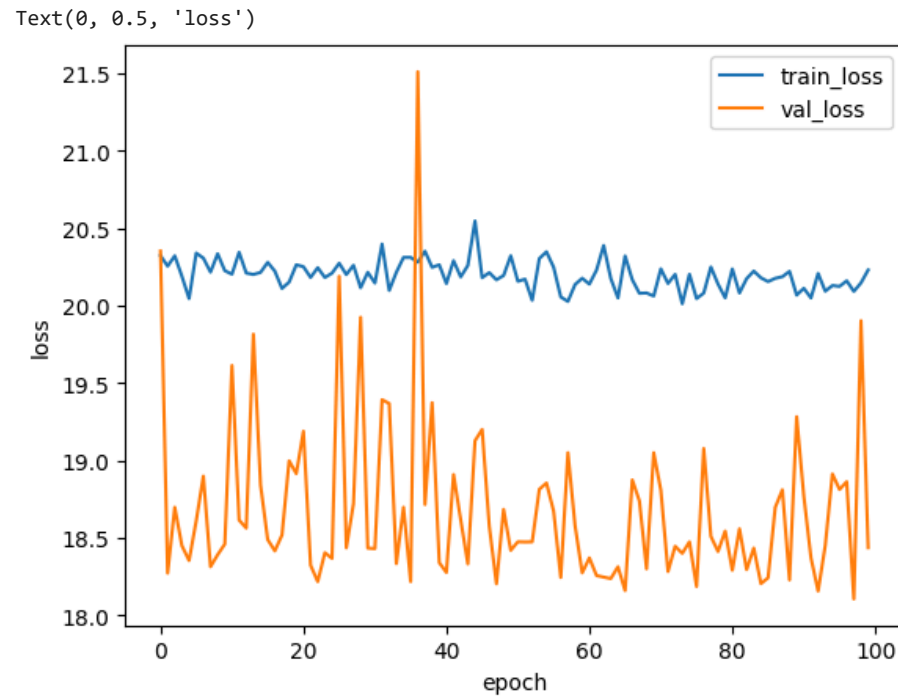
```

```
# This cell is no longer needed as print statement is moved to training loop.
```

```

import matplotlib.pyplot as plt
loss_df=pd.DataFrame(
    {
        "train_loss":train_loss,
        "val_loss":val_loss
    }
)
plt.plot(loss_df["train_loss"],label="train_loss")
plt.plot(loss_df["val_loss"],label="val_loss")
plt.legend()
plt.xlabel("epoch")
plt.ylabel("loss")

```



✓ Saving and Loading the Model

It's good practice to save your trained model so you can reuse it later without retraining. PyTorch models are typically saved by serializing their `state_dict`, which contains all the learnable parameters.

```
model_save_path = 'powerplant_ann_model.pth'
torch.save(model.state_dict(), model_save_path)
print(f"Model saved to {model_save_path}")
```

Model saved to powerplant_ann_model.pth

To load the model, you first need to re-instantiate the model architecture (the `ANN` class in this case) and then load the saved state dictionary into it.

```
# Instantiate a new model to load the saved weights into
loaded_model = ANN()

# Load the state dictionary
loaded_model.load_state_dict(torch.load(model_save_path))
```

```
# Set the loaded model to evaluation mode
loaded_model.eval()

print("Model loaded successfully!")

# You can now use 'loaded_model' for predictions
# For example, let's make a prediction on the first test sample
with torch.no_grad():
    sample_prediction = loaded_model(x_test_tensor[0].unsqueeze(0))
print(f"Prediction for first test sample: {sample_prediction.item():.2f}")
```

```
Model loaded successfully!
Prediction for first test sample: 435.11
Actual value for first test sample: 433.27
```

✓ Model Evaluation

Now, let's evaluate the performance of our loaded model on the test dataset. We'll use the Mean Squared Error (MSE) as our evaluation metric, which was also used as the loss function during training.

```
loaded_model.eval() # Set the model to evaluation mode
with torch.no_grad(): # Disable gradient calculations
    test_predictions = loaded_model(x_test_tensor)
    test_loss = loss_fn(test_predictions, y_test_tensor)
    train_mse_loss=loss_fn(model(x_train_tensor),y_train_tensor)
    print(f"Mean Squared Error on the train set: {train_mse_loss.item():.2f}")

print(f"Mean Squared Error on the test set: {test_loss.item():.2f}")
```

```
Mean Squared Error on the train set: 19.86
Mean Squared Error on the test set: 18.44
```

```
from sklearn.metrics import r2_score
r2_score(y_test,test_predictions.numpy())
```

```
0.9355534560421992
```

Start coding or [generate](#) with AI.